



# Advanced Encryption Schemes

Cecilia Boschini and Kristina Hostáková

# Outline

Recap: Lecture 1

Proof: IND-CPA implies Semantic Security

ElGamal Encryption

Recap: Cyclic Groups

Active Adversaries

# Public-Key Encryption Scheme

PKE : Syntax (Def. 1.1) + Correctness (Def. 1.2)

A PKE scheme  $\Pi$  with message space  $\mathcal{M}$  is a triple of PPT algorithms (Gen, Enc, Dec):

- $\text{Gen}(1^\lambda) \rightarrow (\textcolor{blue}{pk}, \textcolor{violet}{sk})$  public / secret key pair
- $\text{Enc}(\textcolor{blue}{pk}, m) \rightarrow ct$   $m \in \mathcal{M}$
- $\text{Dec}(\textcolor{violet}{sk}, ct) \rightarrow m \in \mathcal{M} \text{ or } \perp$   $\perp$  signals decryption failure

# Public-Key Encryption Scheme

PKE : Syntax (Def. 1.1) + Correctness (Def. 1.2)

A PKE scheme  $\Pi$  with message space  $\mathcal{M}$  is a triple of PPT algorithms (Gen, Enc, Dec):

- $\text{Gen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$  public / secret key pair
- $\text{Enc}(\text{pk}, m) \rightarrow ct$   $m \in \mathcal{M}$
- $\text{Dec}(\text{sk}, ct) \rightarrow m \in \mathcal{M} \text{ or } \perp$   $\perp$  signals decryption failure

*Perfect correctness:*  $\forall \lambda \in \mathbb{N} \quad \forall (\text{pk}, \text{sk}) \in \text{Gen}(1^\lambda) \quad \forall m \in \mathcal{M}$ :

$$\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) = m] = 1.$$

# Security

Intuition:  $ct$  **leaks nothing** about the plaintext — **except its length.**

# Security

Intuition:  $ct$  **leaks nothing** about the plaintext — **except its length**.

- **Perfect security** (information-theoretic)
  - Adversary: *unbounded* computing power
  - **Impossible** for public-key encryption (see exercises)

# Security

Intuition:  $ct$  **leaks nothing** about the plaintext — **except its length**.

- **Perfect security** (information-theoretic)
  - Adversary: *unbounded* computing power
  - **Impossible** for public-key encryption (see exercises)
- **Semantic security** (Goldwasser–Micali '82) — *simulation-based*
  - Adversary: *PPT*
  - Goal: learn any non-trivial information about  $m$  from  $ct$
  - Close to the intuition, but cumbersome to work with

# Security

Intuition:  $ct$  **leaks nothing** about the plaintext — **except its length**.

- **Perfect security** (information-theoretic)
  - Adversary: *unbounded* computing power
  - **Impossible** for public-key encryption (see exercises)
- **Semantic security** (Goldwasser–Micali '82) — *simulation-based*
  - Adversary: *PPT*
  - Goal: learn any non-trivial information about  $m$  from  $ct$
  - Close to the intuition, but cumbersome to work with
- **IND-CPA** — *game-based*
  - Security defined via an experiment between  $\mathcal{A}$  and  $\text{Chal}$
  - Easier to work with  $\Rightarrow$  our working definition in this course

# Security

Intuition:  $ct$  **leaks nothing** about the plaintext — **except its length**.

- **Perfect security** (information-theoretic)
  - Adversary: *unbounded* computing power
  - **Impossible** for public-key encryption (see exercises)
- **Semantic security** (Goldwasser–Micali '82) — *simulation-based*
  - Adversary: *PPT*
  - Goal: learn any non-trivial information about  $m$  from  $ct$
  - Close to the intuition, but cumbersome to work with
- **IND-CPA** — *game-based*
  - Security defined via an experiment between  $\mathcal{A}$  and  $\text{Chal}$
  - Easier to work with  $\Rightarrow$  our working definition in this course

**Good news:** the last two are **equivalent**.

# Semantic Security

Idea: *whatever  $\mathcal{A}$  can compute from  $ct$ , it could already compute without  $ct$ .*

Real:  $\epsilon_{\mathcal{A}}$

$(pk, sk) \leftarrow \text{Gen}(1^\lambda)$

$(m, h) \leftarrow \text{Samp}(pk)$

$x \leftarrow \mathcal{A}(pk, h, \text{Enc}(pk, m))$

$\epsilon_{\mathcal{A}} := \Pr[x = f(m, h)]$

Ideal:  $\epsilon_{\mathcal{A}'}$

$(pk, sk) \leftarrow \text{Gen}(1^\lambda)$

$(m, h) \leftarrow \text{Samp}(pk)$

$x \leftarrow \mathcal{A}'(pk, h, |m|)$

$\epsilon_{\mathcal{A}'} := \Pr[x = f(m, h)]$

# Semantic Security

Idea: *whatever  $\mathcal{A}$  can compute from  $ct$ , it could already compute without  $ct$ .*

Real:  $\epsilon_{\mathcal{A}}$

$(pk, sk) \leftarrow \text{Gen}(1^\lambda)$   
 $(m, h) \leftarrow \text{Samp}(pk)$   
 $x \leftarrow \mathcal{A}(pk, h, \text{Enc}(pk, m))$   
 $\epsilon_{\mathcal{A}} := \Pr[x = f(m, h)]$

Ideal:  $\epsilon_{\mathcal{A}'}$

$(pk, sk) \leftarrow \text{Gen}(1^\lambda)$   
 $(m, h) \leftarrow \text{Samp}(pk)$   
 $x \leftarrow \mathcal{A}'(pk, h, |m|)$   
 $\epsilon_{\mathcal{A}'} := \Pr[x = f(m, h)]$

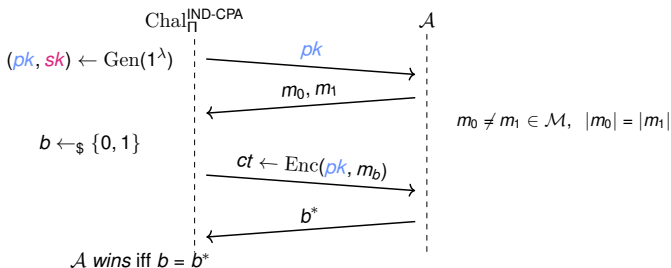
Def. 1.3:  $\Pi$  is *semantically secure* if

$$\forall \text{ PPT } \mathcal{A} \quad \exists \text{ PPT } \mathcal{A}' \quad \forall \text{ PPT Samp} \quad \forall \text{ PT } f: \quad |\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda).$$

Note the quantifier order: the *simulator*  $\mathcal{A}'$  may depend on  $\mathcal{A}$ , but **not** on  $\text{Samp}$  or  $f$ .

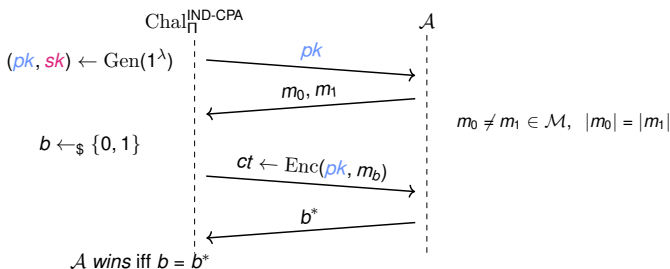
# IND-CPA Security

Game-based definition:  $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$  plays against a challenger  $\text{Chal}_{\Pi}^{\text{IND-CPA}}$ .



# IND-CPA Security

Game-based definition:  $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$  plays against a challenger  $\text{Chal}_{\Pi}^{\text{IND-CPA}}$ .



Def. 1.4:  $\Pi$  is **IND-CPA secure** if for all PPT  $\mathcal{A}$  the advantage

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CPA}}(\lambda) := 2 \cdot \left| \frac{1}{2} - \Pr[\mathcal{A} \text{ wins}] \right| \leq \text{negl}(\lambda).$$

# Where we stand

## Theorem 1.1

$\Pi$  is IND-CPA secure  $\iff \Pi$  is semantically secure.

- **Part I (done):** Semantic security  $\Rightarrow$  IND-CPA

Proved by *contrapositive*:  $\neg \text{IND-CPA} \Rightarrow \neg \text{Semantic security}$ .

# Where we stand

## Theorem 1.1

$\Pi$  is IND-CPA secure  $\iff \Pi$  is semantically secure.

- **Part I (done):** Semantic security  $\Rightarrow$  IND-CPA

Proved by *contrapositive*:  $\neg \text{IND-CPA} \Rightarrow \neg \text{Semantic security}$ .

- **Part II (today):** IND-CPA  $\Rightarrow$  Semantic security

A *direct* proof — with a reduction hidden inside.

# Proof structure: discharging the quantifiers

Recall what we must prove:

$$\forall \text{ PPT } \mathcal{A} \quad \exists \text{ PPT } \mathcal{A}' \quad \forall \text{ PPT Samp} \quad \forall \text{ PT } f: \quad |\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda)$$

- ❶ Assume  $\Pi$  is *IND-CPA secure*.

# Proof structure: discharging the quantifiers

Recall what we must prove:

$$\forall \text{ PPT } \mathcal{A} \quad \exists \text{ PPT } \mathcal{A}' \quad \forall \text{ PPT Samp} \quad \forall \text{ PT } f: \quad |\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda)$$

- 1 Assume  $\Pi$  is *IND-CPA secure*.
- 2 Let  $\mathcal{A}$  be an *arbitrary* PPT adversary.

( $\forall$ -introduction)

# Proof structure: discharging the quantifiers

Recall what we must prove:

$$\forall \text{ PPT } \mathcal{A} \quad \exists \text{ PPT } \mathcal{A}' \quad \forall \text{ PPT Samp} \quad \forall \text{ PT } f: \quad |\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda)$$

- 1 Assume  $\Pi$  is *IND-CPA secure*.
- 2 Let  $\mathcal{A}$  be an *arbitrary* PPT adversary. ( $\forall$ -introduction)
- 3 **Construct** a PPT simulator  $\mathcal{A}'$ , depending *only on*  $\mathcal{A}$ . ( $\exists$ -introduction)

# Proof structure: discharging the quantifiers

Recall what we must prove:

$$\forall \text{ PPT } \mathcal{A} \quad \exists \text{ PPT } \mathcal{A}' \quad \forall \text{ PPT Samp} \quad \forall \text{ PT } f: \quad |\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda)$$

- 1 Assume  $\Pi$  is *IND-CPA secure*.
- 2 Let  $\mathcal{A}$  be an *arbitrary* PPT adversary. ( $\forall$ -introduction)
- 3 **Construct** a PPT simulator  $\mathcal{A}'$ , depending *only on*  $\mathcal{A}$ . ( $\exists$ -introduction)
- 4 Let Samp and  $f$  be *arbitrary*. ( $\forall$ -introduction)

# Proof structure: discharging the quantifiers

Recall what we must prove:

$$\forall \text{ PPT } \mathcal{A} \quad \exists \text{ PPT } \mathcal{A}' \quad \forall \text{ PPT Samp} \quad \forall \text{ PT } f: \quad |\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda)$$

- ➊ Assume  $\Pi$  is *IND-CPA secure*.
- ➋ Let  $\mathcal{A}$  be an *arbitrary* PPT adversary. (∀-introduction)
- ➌ **Construct** a PPT simulator  $\mathcal{A}'$ , depending *only on*  $\mathcal{A}$ . (∃-introduction)
- ➍ Let Samp and  $f$  be *arbitrary*. (∀-introduction)
- ➎ Show  $|\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda)$ . (here we use 1., by contraposition)

# Proof structure: discharging the quantifiers

Recall what we must prove:

$$\forall \text{ PPT } \mathcal{A} \quad \exists \text{ PPT } \mathcal{A}' \quad \forall \text{ PPT Samp} \quad \forall \text{ PT } f: \quad |\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda)$$

- ➊ Assume  $\Pi$  is *IND-CPA secure*.
  - ➋ Let  $\mathcal{A}$  be an *arbitrary* PPT adversary. (∀-introduction)
  - ➌ **Construct** a PPT simulator  $\mathcal{A}'$ , depending *only on*  $\mathcal{A}$ . (∃-introduction)
  - ➍ Let Samp and  $f$  be *arbitrary*. (∀-introduction)
  - ➎ Show  $|\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda)$ . (here we use 1., by contraposition)
- Contraposition is used *only inside step 5* — the proof as a whole is direct

## Step 2+3: constructing the simulator

Fix arbitrary PPT  $\mathcal{A}$ , construct  $\mathcal{A}'$ :

- If  $\mathcal{A}'$  could produce  $ct \leftarrow \text{Enc}(pk, m)$ , it would simply run  $\mathcal{A}$  and be done.

## Step 2+3: constructing the simulator

Fix arbitrary PPT  $\mathcal{A}$ , construct  $\mathcal{A}'$ :

- If  $\mathcal{A}'$  could produce  $ct \leftarrow \text{Enc}(pk, m)$ , it would simply run  $\mathcal{A}$  and be done.
- But  $\mathcal{A}'$  gets only  $pk, h, |m|$  — it does **not** know  $m$ .

## Step 2+3: constructing the simulator

Fix arbitrary PPT  $\mathcal{A}$ , construct  $\mathcal{A}'$ :

- If  $\mathcal{A}'$  could produce  $ct \leftarrow \text{Enc}(pk, m)$ , it would simply run  $\mathcal{A}$  and be done.
- But  $\mathcal{A}'$  gets only  $pk, h, |m|$  — it does **not** know  $m$ .
- *Idea*: feed  $\mathcal{A}$  a ciphertext of the *wrong* message, of the right length.

Simulator  $\mathcal{A}'(pk, h, |m|)$

1.  $ct' \leftarrow \text{Enc}(pk, 0^{|m|})$
2. Output  $\mathcal{A}(pk, h, ct')$

//  $\mathcal{A}$  is run on *inconsistent*  $h$  and  $ct'$ !

## Step 2+3: constructing the simulator

Fix arbitrary PPT  $\mathcal{A}$ , construct  $\mathcal{A}'$ :

- If  $\mathcal{A}'$  could produce  $ct \leftarrow \text{Enc}(pk, m)$ , it would simply run  $\mathcal{A}$  and be done.
- But  $\mathcal{A}'$  gets only  $pk, h, |m|$  — it does **not** know  $m$ .
- *Idea*: feed  $\mathcal{A}$  a ciphertext of the **wrong** message, of the right length.

Simulator  $\mathcal{A}'(pk, h, |m|)$

1.  $ct' \leftarrow \text{Enc}(pk, 0^{|m|})$
2. Output  $\mathcal{A}(pk, h, ct')$

//  $\mathcal{A}$  is run on **inconsistent**  $h$  and  $ct'$ !

- $\mathcal{A}'$  is PPT because  $\mathcal{A}$  and  $\text{Enc}$  are
- $\mathcal{A}'$  uses **neither**  $\text{Samp}$  **nor**  $f$

## Steps 4–5: the two experiments

Fix arbitrary PPT  $\text{Samp}$  and PT computable  $f$ .

Real:  $\epsilon_{\mathcal{A}}$

$(pk, sk) \leftarrow \text{Gen}(1^\lambda)$

$(m, h) \leftarrow \text{Samp}(pk)$

$ct \leftarrow \text{Enc}(pk, m)$

$x \leftarrow \mathcal{A}(pk, h, ct)$

$\epsilon_{\mathcal{A}} := \Pr[x = f(m, h)]$

Ideal:  $\epsilon_{\mathcal{A}'}$

$(pk, sk) \leftarrow \text{Gen}(1^\lambda)$

$(m, h) \leftarrow \text{Samp}(pk)$

$ct' \leftarrow \text{Enc}(pk, 0^{|m|})$

$x \leftarrow \mathcal{A}(pk, h, ct')$

$\epsilon_{\mathcal{A}'} := \Pr[x = f(m, h)]$

## Steps 4–5: the two experiments

Fix arbitrary PPT  $\text{Samp}$  and PT computable  $f$ .

Real:  $\epsilon_{\mathcal{A}}$

$(pk, sk) \leftarrow \text{Gen}(1^\lambda)$

$(m, h) \leftarrow \text{Samp}(pk)$

$ct \leftarrow \text{Enc}(pk, m)$

$x \leftarrow \mathcal{A}(pk, h, ct)$

$\epsilon_{\mathcal{A}} := \Pr[x = f(m, h)]$

Ideal:  $\epsilon_{\mathcal{A}'}$

$(pk, sk) \leftarrow \text{Gen}(1^\lambda)$

$(m, h) \leftarrow \text{Samp}(pk)$

$ct' \leftarrow \text{Enc}(pk, 0^{|m|})$

$x \leftarrow \mathcal{A}(pk, h, ct')$

$\epsilon_{\mathcal{A}'} := \Pr[x = f(m, h)]$

It remains to show:  $|\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}| \leq \text{negl}(\lambda)$ .

## Step 5: reduction to IND-CPA

By contraposition: assume  $|\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}|$  is **non-negligible** and build  $\mathcal{D}$  against IND-CPA.

$\mathcal{D}(pk)$  -- with  $\mathcal{A}, \text{Samp}, f$  hard-wired

1.  $(m, h) \leftarrow \text{Samp}(pk)$
2. Send  $m_0 := m, m_1 := 0^{|m|}$  to the challenger (note  $|m_0| = |m_1|$ )
3. Receive  $ct \leftarrow \text{Enc}(pk, m_b)$
4.  $x \leftarrow \mathcal{A}(pk, h, ct)$
5. If  $x = f(m, h)$  output  $b^* := 0$ , else output  $b^* := 1$

## Step 5: reduction to IND-CPA

By contraposition: assume  $|\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}|$  is **non-negligible** and build  $\mathcal{D}$  against IND-CPA.

$\mathcal{D}(pk)$  -- with  $\mathcal{A}, \text{Samp}, f$  hard-wired

1.  $(m, h) \leftarrow \text{Samp}(pk)$
2. Send  $m_0 := m, m_1 := 0^{|m|}$  to the challenger (note  $|m_0| = |m_1|$ )
3. Receive  $ct \leftarrow \text{Enc}(pk, m_b)$
4.  $x \leftarrow \mathcal{A}(pk, h, ct)$
5. If  $x = f(m, h)$  output  $b^* := 0$ , else output  $b^* := 1$

- $\mathcal{D}$  is PPT *because Samp is PPT and  $f$  is efficiently computable*

## Step 5: finishing the computation

- $b = 0$ :  $\mathcal{D}$  simulates *Real*  $\Pr[\mathcal{D} \text{ wins} \mid b = 0] = \epsilon_{\mathcal{A}}$
- $b = 1$ :  $\mathcal{D}$  simulates *Ideal*  $\Pr[\mathcal{D} \text{ wins} \mid b = 1] = 1 - \epsilon_{\mathcal{A}'}$

Hence, we have

$$\begin{aligned}\Pr[\mathcal{D} \text{ wins}] &= \Pr[\mathcal{D} \text{ wins} \mid b = 0] \cdot \Pr[b = 0] + \Pr[\mathcal{D} \text{ wins} \mid b = 1] \cdot \Pr[b = 1] \\ &= \frac{1}{2} \epsilon_{\mathcal{A}} + \frac{1}{2} (1 - \epsilon_{\mathcal{A}'}) = \frac{1}{2} (1 + \epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'})\end{aligned}$$

## Step 5: finishing the computation

- $b = 0$ :  $\mathcal{D}$  simulates *Real*  $\Pr[\mathcal{D} \text{ wins} \mid b = 0] = \epsilon_{\mathcal{A}}$
- $b = 1$ :  $\mathcal{D}$  simulates *Ideal*  $\Pr[\mathcal{D} \text{ wins} \mid b = 1] = 1 - \epsilon_{\mathcal{A}'}$

Hence, we have

$$\begin{aligned}\Pr[\mathcal{D} \text{ wins}] &= \Pr[\mathcal{D} \text{ wins} \mid b = 0] \cdot \Pr[b = 0] + \Pr[\mathcal{D} \text{ wins} \mid b = 1] \cdot \Pr[b = 1] \\ &= \frac{1}{2} \epsilon_{\mathcal{A}} + \frac{1}{2} (1 - \epsilon_{\mathcal{A}'}) = \frac{1}{2} (1 + \epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'})\end{aligned}$$

That means

$$\text{Adv}_{\Pi, \mathcal{D}}^{\text{IND-CPA}}(\lambda) = 2 \left| \frac{1}{2} - \Pr[\mathcal{D} \text{ wins}] \right| = 2 \left| \frac{1}{2} - \frac{1}{2} (1 + \epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}) \right| = |\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}|$$

## Step 5: finishing the computation

- $b = 0$ :  $\mathcal{D}$  simulates *Real*  $\Pr[\mathcal{D} \text{ wins} \mid b = 0] = \epsilon_{\mathcal{A}}$
- $b = 1$ :  $\mathcal{D}$  simulates *Ideal*  $\Pr[\mathcal{D} \text{ wins} \mid b = 1] = 1 - \epsilon_{\mathcal{A}'}$

Hence, we have

$$\begin{aligned}\Pr[\mathcal{D} \text{ wins}] &= \Pr[\mathcal{D} \text{ wins} \mid b = 0] \cdot \Pr[b = 0] + \Pr[\mathcal{D} \text{ wins} \mid b = 1] \cdot \Pr[b = 1] \\ &= \frac{1}{2} \epsilon_{\mathcal{A}} + \frac{1}{2} (1 - \epsilon_{\mathcal{A}'}) = \frac{1}{2} (1 + \epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'})\end{aligned}$$

That means

$$\text{Adv}_{\Pi, \mathcal{D}}^{\text{IND-CPA}}(\lambda) = 2 \left| \frac{1}{2} - \Pr[\mathcal{D} \text{ wins}] \right| = 2 \left| \frac{1}{2} - \frac{1}{2} (1 + \epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}) \right| = |\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}|$$

So  $|\epsilon_{\mathcal{A}} - \epsilon_{\mathcal{A}'}|$  non-negligible  $\Rightarrow \text{Adv}_{\Pi, \mathcal{D}}^{\text{IND-CPA}}$  non-negligible.

Contradiction with 1.

# Takeaways

- Semantic security and IND-CPA are **equivalent**

⇒ from now on we only work with IND-CPA.

# Takeaways

- Semantic security and IND-CPA are **equivalent**

⇒ from now on we only work with IND-CPA.

- The simulator was **explicit**: encrypt  $0^{|m|}$  instead of  $m$ .

Common pattern: “the simulator does the honest thing to a dummy input”.

# Takeaways

- Semantic security and IND-CPA are **equivalent**

⇒ from now on we only work with IND-CPA.

- The simulator was **explicit**: encrypt  $0^{|m|}$  instead of  $m$ .

Common pattern: “the simulator does the honest thing to a dummy input”.

- The reduction  $\mathcal{D}$  needed  $\text{Samp}$  and  $f$  to be **efficient** — otherwise  $\mathcal{D}$  is not PPT.

## Takeaways

- Semantic security and IND-CPA are **equivalent**

⇒ from now on we only work with IND-CPA.

- The simulator was **explicit**: encrypt  $0^{|m|}$  instead of  $m$ .

Common pattern: “the simulator does the honest thing to a dummy input”.

- The reduction  $\mathcal{D}$  needed  $\text{Samp}$  and  $f$  to be **efficient** — otherwise  $\mathcal{D}$  is not PPT.
- Note where the quantifiers went:
  - $\mathcal{A}'$  was built from  $\mathcal{A}$  **alone**
  - $\text{Samp}$ ,  $f$  only entered later, absorbed into the reduction  $\mathcal{D}$

# Outline

Recap: Lecture 1

Proof: IND-CPA implies Semantic Security

ElGamal Encryption

Recap: Cyclic Groups

Active Adversaries

## Cyclic groups: what we need

- $\mathbb{G}$  cyclic group of *prime order*  $p$  with generator  $g$ :

$$\mathbb{G} = \langle g \rangle = \{1, g, g^2, \dots, g^{p-1}\}$$

Prime order  $\Rightarrow$  *every*  $h \neq 1$  is a generator.

## Cyclic groups: what we need

- $\mathbb{G}$  cyclic group of *prime order*  $p$  with generator  $g$ :

$$\mathbb{G} = \langle g \rangle = \{1, g, g^2, \dots, g^{p-1}\}$$

Prime order  $\Rightarrow$  *every*  $h \neq 1$  is a generator.

- Written multiplicatively  $(\mathbb{G}, \cdot, 1)$  or additively  $(\mathbb{G}, +, 0)$ ; mostly we just say “group  $\mathbb{G}$ ”.

## Cyclic groups: what we need

- $\mathbb{G}$  cyclic group of *prime order*  $p$  with generator  $g$ :

$$\mathbb{G} = \langle g \rangle = \{1, g, g^2, \dots, g^{p-1}\}$$

Prime order  $\Rightarrow$  *every*  $h \neq 1$  is a generator.

- Written multiplicatively  $(\mathbb{G}, \cdot, 1)$  or additively  $(\mathbb{G}, +, 0)$ ; mostly we just say “group  $\mathbb{G}$ ”.
- As a group,  $\mathbb{G}$  is *nothing but*  $\mathbb{Z}_p$ :

$$(\mathbb{G}, \cdot, 1) \cong (\mathbb{Z}_p, + \bmod p, 0)$$

... but the isomorphism is **hard to compute** — that is the whole point.

## Cyclic groups: what we need

- $\mathbb{G}$  cyclic group of *prime order*  $p$  with generator  $g$ :

$$\mathbb{G} = \langle g \rangle = \{1, g, g^2, \dots, g^{p-1}\}$$

Prime order  $\Rightarrow$  *every*  $h \neq 1$  is a generator.

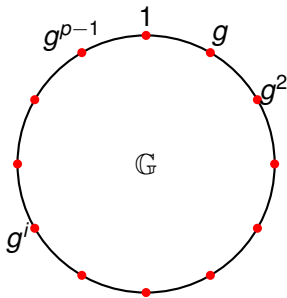
- Written multiplicatively  $(\mathbb{G}, \cdot, 1)$  or additively  $(\mathbb{G}, +, 0)$ ; mostly we just say “group  $\mathbb{G}$ ”.
- As a group,  $\mathbb{G}$  is *nothing but*  $\mathbb{Z}_p$ :

$$(\mathbb{G}, \cdot, 1) \cong (\mathbb{Z}_p, + \bmod p, 0)$$

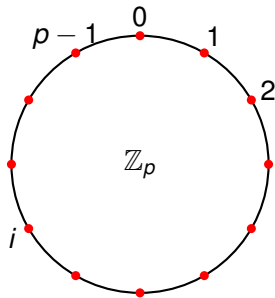
... but the isomorphism is **hard to compute** — that is the whole point.

- Candidates: elliptic curves ( $\approx 250$  bits/element), subgroups of  $\mathbb{F}_q^*$  ( $\approx 2000$ – $4000$  bits/element).

## Two views of the same group

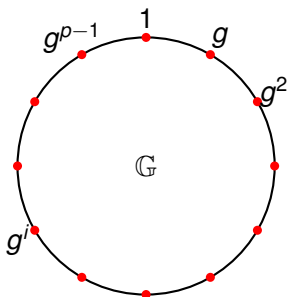


$(\mathbb{G} = \langle g \rangle, \cdot, 1)$ , prime order  $p$   
 “computation in the group”

 $\cong$ 


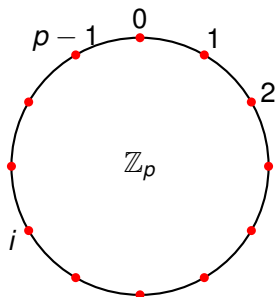
$(\mathbb{Z}_p = \langle 1 \rangle, + \bmod p, 0)$   
 “computation in the exponent”

## Two views of the same group



$(\mathbb{G} = \langle g \rangle, \cdot, 1)$ , prime order  $p$   
 “computation in the group”

$\cong$



$(\mathbb{Z}_p = \langle 1 \rangle, + \bmod p, 0)$   
 “computation in the exponent”

Going  $\mathbb{G} \rightarrow \mathbb{Z}_p$  is the **discrete logarithm** — believed hard.

## Three hardness assumptions

Let  $\mathbb{G} = \langle g \rangle$  be cyclic of prime order  $p$ , and  $x, y, z \leftarrow_{\$} \mathbb{Z}_p$ .

## Three hardness assumptions

Let  $\mathbb{G} = \langle g \rangle$  be cyclic of prime order  $p$ , and  $x, y, z \leftarrow_{\$} \mathbb{Z}_p$ .

$\mathbb{G}$  and  $p$  depend on  $\lambda$ ; we keep this implicit.

- *DLog*: given  $g^x$ , find  $x$ .

## Three hardness assumptions

Let  $\mathbb{G} = \langle g \rangle$  be cyclic of prime order  $p$ , and  $x, y, z \leftarrow_{\$} \mathbb{Z}_p$ .

$\mathbb{G}$  and  $p$  depend on  $\lambda$ ; we keep this implicit.

- *DLog*: given  $g^x$ , find  $x$ .
- *CDH*: given  $g^x, g^y$ , compute  $g^{xy}$ .

## Three hardness assumptions

Let  $\mathbb{G} = \langle g \rangle$  be cyclic of prime order  $p$ , and  $x, y, z \leftarrow_{\$} \mathbb{Z}_p$ .

$\mathbb{G}$  and  $p$  depend on  $\lambda$ ; we keep this implicit.

- **DLog**: given  $g^x$ , find  $x$ .
- **CDH**: given  $g^x, g^y$ , compute  $g^{xy}$ .
- **DDH**: distinguish  $(g^x, g^y, g^{xy})$  from  $(g^x, g^y, g^z)$ .

## Three hardness assumptions

Let  $\mathbb{G} = \langle g \rangle$  be cyclic of prime order  $p$ , and  $x, y, z \leftarrow_{\$} \mathbb{Z}_p$ .

$\mathbb{G}$  and  $p$  depend on  $\lambda$ ; we keep this implicit.

- **DLog**: given  $g^x$ , find  $x$ .
- **CDH**: given  $g^x, g^y$ , compute  $g^{xy}$ .
- **DDH**: distinguish  $(g^x, g^y, g^{xy})$  from  $(g^x, g^y, g^z)$ .

**DDH hard**  $\longrightarrow$  **CDH hard**  $\longrightarrow$  **DLog hard**

Converses: CDH  $\not\Rightarrow$  DDH (DDH is **easy** in pairing groups); DLog  $\not\Rightarrow$  CDH is open.

# The assumption we will use

## Definition 2.1: DDH assumption

The *DDH assumption* holds in  $\mathbb{G}$  if for all PPT  $\mathcal{A}$  the advantage

$$\text{Adv}_{\mathbb{G}, \mathcal{A}}^{\text{DDH}}(\lambda) := 2 \cdot \left| \frac{1}{2} - \Pr \left[ \begin{array}{l} b \leftarrow_{\$} \{0, 1\} \\ x, y, z_1 \leftarrow_{\$} \mathbb{Z}_p \\ z_0 := x \cdot y \bmod p \end{array} : \mathcal{A}(1^\lambda, g, g^x, g^y, g^{z_b}) = b \right] \right|$$

is negligible in  $\lambda$ .

# The assumption we will use

## Definition 2.1: DDH assumption

The *DDH assumption* holds in  $\mathbb{G}$  if for all PPT  $\mathcal{A}$  the advantage

$$\text{Adv}_{\mathbb{G}, \mathcal{A}}^{\text{DDH}}(\lambda) := 2 \cdot \left| \frac{1}{2} - \Pr \left[ \begin{array}{l} b \leftarrow_{\$} \{0, 1\} \\ x, y, z_1 \leftarrow_{\$} \mathbb{Z}_p \\ z_0 := x \cdot y \bmod p \end{array} : \mathcal{A}(1^\lambda, g, g^x, g^y, g^{z_b}) = b \right] \right|$$

is negligible in  $\lambda$ .

- Equivalently:  $(g, g^x, g^y, g^{xy})$  looks like  $(g, g^x, g^y, g^z)$  —  *$g^{xy}$  is as good as a random group element.*
- **Next:** this is exactly what makes **ElGamal** IND-CPA secure.

# ElGamal Encryption Scheme

Public parameters: a cyclic group  $= \langle g \rangle$  of prime order  $p$ .

Message space  $\mathcal{M} = \mathbb{G}$ .

$\text{Gen}(1^\lambda) : \textcolor{violet}{sk} := x \leftarrow_{\$} \mathbb{Z}_p, \quad \textcolor{blue}{pk} := h := g^x$

# ElGamal Encryption Scheme

Public parameters: a cyclic group  $= \langle g \rangle$  of prime order  $p$ .

Message space  $\mathcal{M} = \mathbb{G}$ .

$\text{Gen}(1^\lambda) : \text{sk} := x \leftarrow_{\$} \mathbb{Z}_p, \quad \text{pk} := h := g^x$

$\text{Enc}(\text{pk}, m \in \mathbb{G}) :$

- $r \leftarrow_{\$} \mathbb{Z}_p$
- $c_1 := g^r$
- $c_2 := m \cdot h^r$
- $ct := (c_1, c_2) \in \mathbb{G} \times \mathbb{G}$

# ElGamal Encryption Scheme

Public parameters: a cyclic group  $= \langle g \rangle$  of prime order  $p$ .

Message space  $\mathcal{M} = \mathbb{G}$ .

$\text{Gen}(1^\lambda) : \text{sk} := x \leftarrow_{\$} \mathbb{Z}_p, \quad \text{pk} := h := g^x$

$\text{Enc}(\text{pk}, m \in \mathbb{G}) :$

- $r \leftarrow_{\$} \mathbb{Z}_p$
- $c_1 := g^r$
- $c_2 := m \cdot h^r$
- $ct := (c_1, c_2) \in \mathbb{G} \times \mathbb{G}$

$\text{Dec}(\text{sk}, ct) : \text{Parse } ct = (c_1, c_2), \text{ output } \frac{c_2}{c_1^x}.$

# ElGamal Encryption Scheme

Public parameters: a cyclic group  $= \langle g \rangle$  of prime order  $p$ .

Message space  $\mathcal{M} = \mathbb{G}$ .

$\text{Gen}(1^\lambda) : \textcolor{violet}{sk} := x \leftarrow_{\$} \mathbb{Z}_p, \quad \textcolor{blue}{pk} := h := g^x$

$\text{Enc}(\textcolor{blue}{pk}, m \in \mathbb{G}) :$

- $r \leftarrow_{\$} \mathbb{Z}_p$
- $c_1 := g^r$
- $c_2 := m \cdot h^r$
- $ct := (c_1, c_2) \in \mathbb{G} \times \mathbb{G}$

$\text{Dec}(\textcolor{violet}{sk}, ct) : \text{Parse } ct = (c_1, c_2), \text{ output } \frac{c_2}{c_1^x}.$

⚠  $m$  must be a *group element* — encoding bit strings into  $\mathbb{G}$  is a separate (and annoying) problem. We come back to this with *KEM/DEM*.

## ElGamal - Correctness

Let  $(sk, pk) = (x, h = g^x)$ ,  $m \in \mathbb{G}$  and  $ct = (c_1 = g^r, c_2 = m \cdot h^r)$ .

Then

$$\frac{c_2}{c_1^x} = \frac{m \cdot h^r}{(g^r)^x} = \frac{m \cdot (g^x)^r}{g^{r \cdot x}} = m.$$

## ElGamal - Correctness

Let  $(sk, pk) = (x, h = g^x)$ ,  $m \in \mathbb{G}$  and  $ct = (c_1 = g^r, c_2 = m \cdot h^r)$ .

Then

$$\frac{c_2}{c_1^x} = \frac{m \cdot h^r}{(g^r)^x} = \frac{m \cdot (g^x)^r}{g^{r \cdot x}} = m.$$

So ElGamal satisfies *perfect correctness*.

# ElGamal - Security

Theorem 2.1: IND-CPA security of ElGamal

If the DDH assumption holds in  $\mathbb{G}$ , then ElGamal is IND-CPA secure.

# ElGamal - Security

## Theorem 2.1: IND-CPA security of ElGamal

If the DDH assumption holds in  $\mathbb{G}$ , then ElGamal is IND-CPA secure.

- We will show *real-or-random* IND-CPA:
    - $\mathcal{A}$  chooses  $m_0$ , Chal chooses  $m_1 \leftarrow_{\$} \mathcal{M}$
    - $\mathcal{A}$  gets  $ct \leftarrow \text{Enc}(pk, m_b)$  for  $b \leftarrow_{\$} \{0, 1\}$  and has to guess  $b$
- Equivalent to left-or-right IND-CPA (exercise sheet).

# ElGamal - Security

## Theorem 2.1: IND-CPA security of ElGamal

If the DDH assumption holds in  $\mathbb{G}$ , then ElGamal is IND-CPA secure.

- We will show *real-or-random* IND-CPA:
  - $\mathcal{A}$  chooses  $m_0$ , Chal chooses  $m_1 \leftarrow_{\$} \mathcal{M}$
  - $\mathcal{A}$  gets  $ct \leftarrow \text{Enc}(pk, m_b)$  for  $b \leftarrow_{\$} \{0, 1\}$  and has to guess  $b$Equivalent to left-or-right IND-CPA (exercise sheet).
- It matches the DDH game: “*real object* vs. *random object*”.

## Proof: DDH in $\mathbb{G} \Rightarrow$ IND-CPA ElGamal

### Proof idea:

- Assume a PPT adversary  $\mathcal{A}$  against IND-CPA security of ElGamal

## Proof: DDH in $\mathbb{G} \Rightarrow$ IND-CPA ElGamal

### Proof idea:

- Assume a PPT adversary  $\mathcal{A}$  against IND-CPA security of ElGamal
- Construct  $\mathcal{B}$  against DDH in  $\mathbb{G}$ , running  $\mathcal{A}$  as a *subroutine*

## Proof: DDH in $\mathbb{G} \Rightarrow$ IND-CPA ElGamal

### Proof idea:

- Assume a PPT adversary  $\mathcal{A}$  against IND-CPA security of ElGamal
- Construct  $\mathcal{B}$  against DDH in  $\mathbb{G}$ , running  $\mathcal{A}$  as a *subroutine*
- $\mathcal{B}$  embeds its DDH challenge into the ciphertext given to  $\mathcal{A}$ :
  - real DH tuple  $\rightsquigarrow \mathcal{A}$  sees an encryption of *its own message*
  - random tuple  $\rightsquigarrow \mathcal{A}$  sees an encryption of a *random message*

# Proof: DDH in $\mathbb{G} \Rightarrow$ IND-CPA ElGamal

## Proof idea:

- Assume a PPT adversary  $\mathcal{A}$  against IND-CPA security of ElGamal
- Construct  $\mathcal{B}$  against DDH in  $\mathbb{G}$ , running  $\mathcal{A}$  as a *subroutine*
- $\mathcal{B}$  embeds its DDH challenge into the ciphertext given to  $\mathcal{A}$ :
  - real DH tuple  $\rightsquigarrow \mathcal{A}$  sees an encryption of *its own message*
  - random tuple  $\rightsquigarrow \mathcal{A}$  sees an encryption of a *random message*
- $\mathcal{B}$  simply **echoes  $\mathcal{A}$ 's guess**  $\Rightarrow \text{Adv}_{\mathbb{G}, \mathcal{B}}^{\text{DDH}} = \text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CPA}}$

# Proof: DDH in $\mathbb{G} \Rightarrow \text{IND-CPA ElGamal (1)}$

Chal $_{\mathbb{G}}^{\text{DDH}}$

$\mathcal{B}$

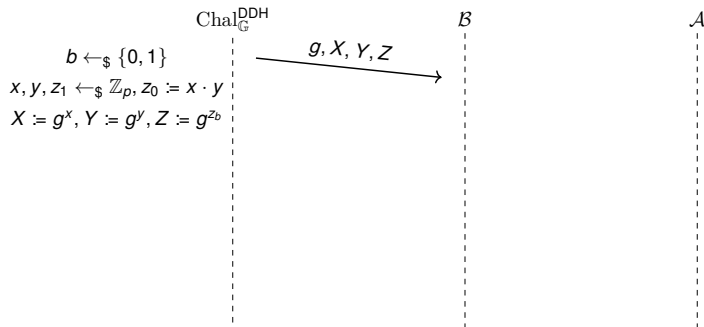
$\mathcal{A}$

⋮

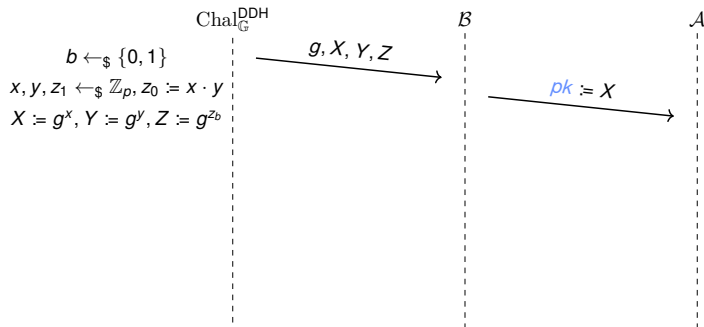
⋮

⋮

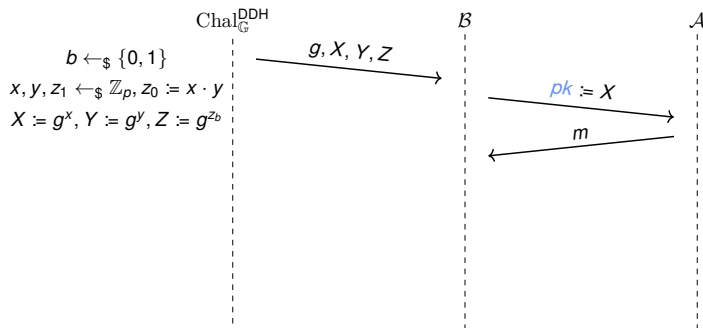
# Proof: DDH in $\mathbb{G} \Rightarrow \text{IND-CPA ElGamal (1)}$



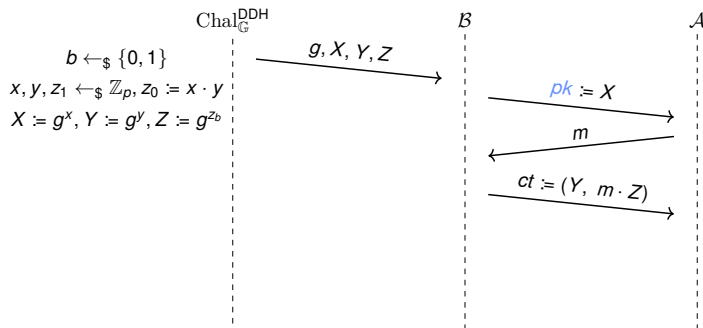
# Proof: DDH in $\mathbb{G} \Rightarrow \text{IND-CPA ElGamal (1)}$



# Proof: DDH in $\mathbb{G} \Rightarrow \text{IND-CPA ElGamal (1)}$

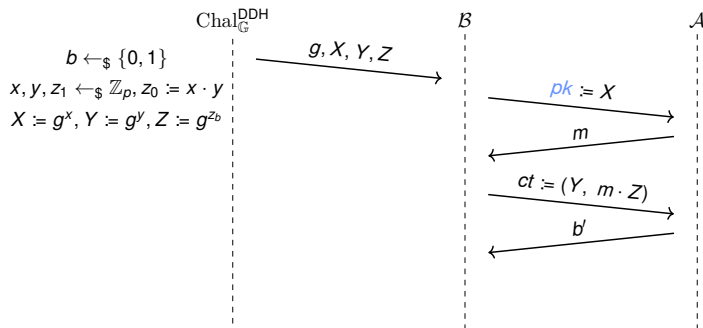


# Proof: DDH in $\mathbb{G} \Rightarrow \text{IND-CPA ElGamal (1)}$



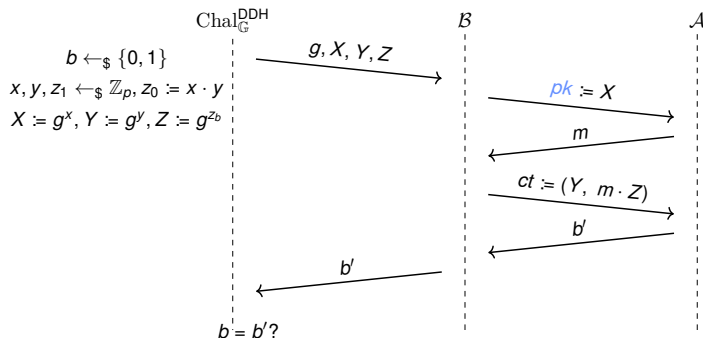
$\mathcal{B}$  never knows  $x$  or  $y$  — it only *forwards group elements*.

# Proof: DDH in $\mathbb{G} \Rightarrow \text{IND-CPA ElGamal (1)}$



$\mathcal{B}$  never knows  $x$  or  $y$  — it only *forwards group elements*.

# Proof: DDH in $\mathbb{G} \Rightarrow \text{IND-CPA ElGamal (1)}$



$\mathcal{B}$  never knows  $x$  or  $y$  — it only *forwards group elements*.

## Proof: DDH in $\mathbb{G} \Rightarrow \text{IND-CPA ElGamal (2)}$

**What does  $\mathcal{A}$  see?** Recall  $pk = X = g^x$  and  $ct = (Y, m \cdot Z) = (g^y, m \cdot Z)$ .

$b = 0$ : real

- $Z = g^{xy}$
- $ct = (g^y, m \cdot g^{xy}) = (g^y, m \cdot pk^y)$
- $ct$  is exactly  $\text{Enc}(pk, m)$  with randomness  $r := y$

# Proof: DDH in $\mathbb{G} \Rightarrow$ IND-CPA ElGamal (2)

**What does  $\mathcal{A}$  see?** Recall  $pk = X = g^x$  and  $ct = (Y, m \cdot Z) = (g^y, m \cdot Z)$ .

$b = 0$ : real

- $Z = g^{xy}$
- $ct = (g^y, m \cdot g^{xy}) = (g^y, m \cdot pk^y)$
- $ct$  is exactly  $\text{Enc}(pk, m)$  with randomness  $r := y$

$b = 1$ : random

- $Z = g^{z_1}$ ,  $z_1$  uniform, indep. of  $x, y$
- $m \cdot Z$  is *uniform in  $\mathbb{G}$* , independent of  $g^y$
- $ct$  has the same distribution as  $\text{Enc}(pk, m_1)$  for  $m_1 \leftarrow_{\$} \mathbb{G}$

## Proof: DDH in $\mathbb{G} \Rightarrow \text{IND-CPA ElGamal (2)}$

**What does  $\mathcal{A}$  see?** Recall  $\textcolor{blue}{pk} = X = g^x$  and  $ct = (Y, m \cdot Z) = (g^y, m \cdot Z)$ .

$b = 0$ : real

- $Z = g^{xy}$
- $ct = (g^y, m \cdot g^{xy}) = (g^y, m \cdot \textcolor{blue}{pk}^y)$
- $ct$  is exactly  $\text{Enc}(\textcolor{blue}{pk}, m)$  with randomness  $r := y$

$b = 1$ : random

- $Z = g^{z_1}$ ,  $z_1$  uniform, indep. of  $x, y$
- $m \cdot Z$  is *uniform in  $\mathbb{G}$* , independent of  $g^y$
- $ct$  has the same distribution as  $\text{Enc}(\textcolor{blue}{pk}, m_1)$  for  $m_1 \leftarrow_{\$} \mathbb{G}$

$\mathcal{B}$  **perfectly simulates** the real-or-random experiment, and its DDH bit  $b$  **is** the challenge bit of that experiment.

## Proof: DDH in $\mathbb{G} \Rightarrow$ IND-CPA ElGamal (3)

Since  $\mathcal{B}$  outputs exactly  $\mathcal{A}$ 's guess  $b'$ :

$$\Pr[\mathcal{B} \text{ wins the DDH experiment}] = \Pr[\mathcal{A} \text{ wins the IND-CPA experiment}]$$

## Proof: DDH in $\mathbb{G} \Rightarrow$ IND-CPA ElGamal (3)

Since  $\mathcal{B}$  outputs exactly  $\mathcal{A}$ 's guess  $b'$ :

$$\Pr[\mathcal{B} \text{ wins the DDH experiment}] = \Pr[\mathcal{A} \text{ wins the IND-CPA experiment}]$$

and therefore

$$\text{Adv}_{\mathbb{G}, \mathcal{B}}^{\text{DDH}}(\lambda) = \text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CPA}}(\lambda).$$

## Proof: DDH in $\mathbb{G} \Rightarrow$ IND-CPA ElGamal (3)

Since  $\mathcal{B}$  outputs exactly  $\mathcal{A}$ 's guess  $b'$ :

$$\Pr[\mathcal{B} \text{ wins the DDH experiment}] = \Pr[\mathcal{A} \text{ wins the IND-CPA experiment}]$$

and therefore

$$\text{Adv}_{\mathbb{G}, \mathcal{B}}^{\text{DDH}}(\lambda) = \text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CPA}}(\lambda).$$

- $\mathcal{A}$  breaks IND-CPA  $\Rightarrow \mathcal{B}$  breaks DDH— contradiction.  $\square$
- $\mathcal{B}$  runs in essentially the *same time* as  $\mathcal{A}$ : the reduction is **tight**.

## Proof: DDH in $\mathbb{G} \Rightarrow$ IND-CPA ElGamal (3)

Since  $\mathcal{B}$  outputs exactly  $\mathcal{A}$ 's guess  $b'$ :

$$\Pr[\mathcal{B} \text{ wins the DDH experiment}] = \Pr[\mathcal{A} \text{ wins the IND-CPA experiment}]$$

and therefore

$$\text{Adv}_{\mathbb{G}, \mathcal{B}}^{\text{DDH}}(\lambda) = \text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CPA}}(\lambda).$$

- $\mathcal{A}$  breaks IND-CPA  $\Rightarrow \mathcal{B}$  breaks DDH— contradiction.  $\square$
- $\mathcal{B}$  runs in essentially the *same time* as  $\mathcal{A}$ : the reduction is **tight**.

*Converse also holds:* if ElGamal is IND-CPA secure, then DDH holds in  $\mathbb{G}$  (exercise).

## ElGamal is malleable

Recall ElGamal:

$pk = h = g^x$ ,  $sk = x$  and ciphertexts are of the form

$$\text{Enc}(pk, m) = ct = (g^r, m \cdot h^r), \text{ for } r \leftarrow \mathbb{Z}_p.$$

- Upon seeing  $ct = (c_1, c_2) \leftarrow \text{Enc}(pk, m)$

## ElGamal is malleable

Recall ElGamal:

$pk = h = g^x$ ,  $sk = x$  and ciphertexts are of the form

$$\text{Enc}(pk, m) = ct = (g^r, m \cdot h^r), \text{ for } r \leftarrow \mathbb{Z}_p.$$

- Upon seeing  $ct = (c_1, c_2) \leftarrow \text{Enc}(pk, m)$
- Anyone can generate  $ct' = (c_1, c_2 \cdot m')$

## ElGamal is malleable

Recall ElGamal:

$pk = h = g^x$ ,  $sk = x$  and ciphertexts are of the form

$$\text{Enc}(pk, m) = ct = (g^r, m \cdot h^r), \text{ for } r \leftarrow \mathbb{Z}_p.$$

- Upon seeing  $ct = (c_1, c_2) \leftarrow \text{Enc}(pk, m)$
- Anyone can generate  $ct' = (c_1, c_2 \cdot m')$
- $\text{Dec}(sk, ct') = m \cdot m'$  because

$$(g^r, m \cdot h^r \cdot m') = (g^r, (m \cdot m')h^r)$$

ElGamal is *multiplicatively homomorphic!*

# Outline

Recap: Lecture 1

Proof: IND-CPA implies Semantic Security

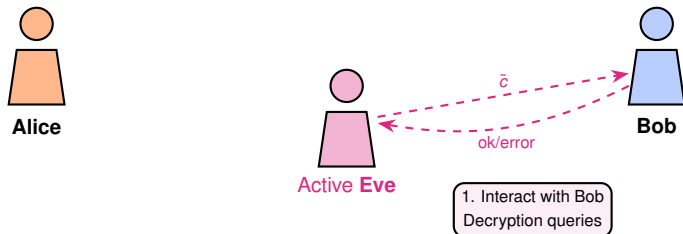
ElGamal Encryption

Recap: Cyclic Groups

Active Adversaries

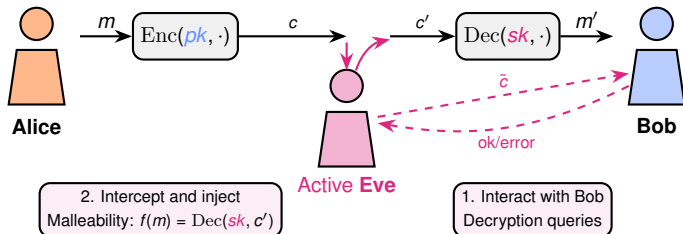
## Recall: active adversaries

- IND-CPA only protects against a *passive* Eve.
- An active Eve can *interact* with Bob and *modify* ciphertexts.



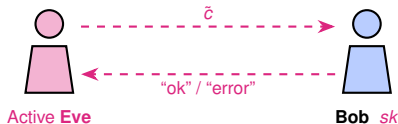
## Recall: active adversaries

- IND-CPA only protects against a *passive* Eve.
- An active Eve can *interact* with Bob and *modify* ciphertexts.



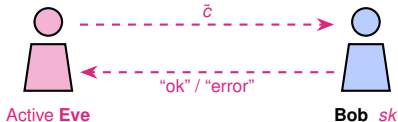
## Decryption oracles in practice

Bob holds *sk* and *reacts* to whatever arrives — that reaction leaks information.



## Decryption oracles in practice

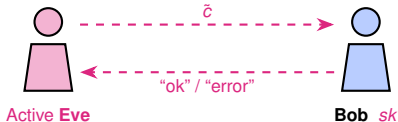
Bob holds *sk* and *reacts* to whatever arrives — that reaction leaks information.



- **Bleichenbacher (1998):**  $\approx 10^6$  such queries break RSA PKCS#1 v1.5 in SSL

## Decryption oracles in practice

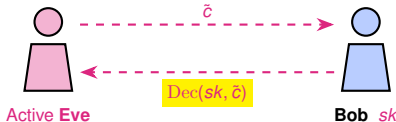
Bob holds *sk* and *reacts* to whatever arrives — that reaction leaks information.



- **Bleichenbacher (1998):**  $\approx 10^6$  such queries break RSA PKCS#1 v1.5 in SSL
- Even *one bit* of feedback per ciphertext is enough

## Decryption oracles in practice

Bob holds *sk* and *reacts* to whatever arrives — that reaction leaks information.



- **Bleichenbacher (1998):**  $\approx 10^6$  such queries break RSA PKCS#1 v1.5 in SSL
- Even *one bit* of feedback per ciphertext is enough
- Model the worst case:  $\mathcal{A}$  gets a full decryption oracle  $\mathcal{O}_{\text{Dec}}(\cdot) = \text{Dec}(sk, \cdot)$ .

## First attempt: lunchtime attacks

Naor–Yung 1990:  $\mathcal{A}$  has *temporary* access to the decryption device.  
(breaks into the office over lunch; a decryption service later shut down)



## First attempt: lunchtime attacks

Naor–Yung 1990:  $\mathcal{A}$  has *temporary* access to the decryption device.  
(breaks into the office over lunch; a decryption service later shut down)



$\Rightarrow$  **IND-CCA1** ( $q_D = \text{poly}(\lambda)$  adaptive queries, all *before* the challenge)  
Indistinguishability Under **Chosen Ciphertext Attack**

# IND-CCA1 security experiment

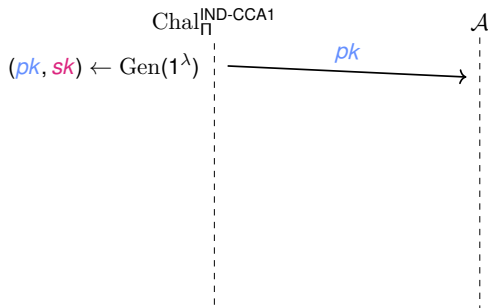
Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.

$\text{Chal}_{\Pi}^{\text{IND-CCA1}}$

$\mathcal{A}$

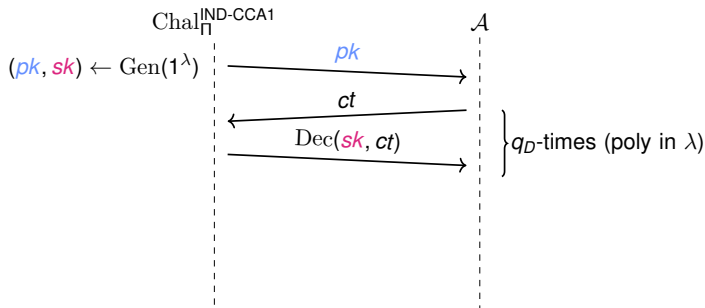
# IND-CCA1 security experiment

Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.



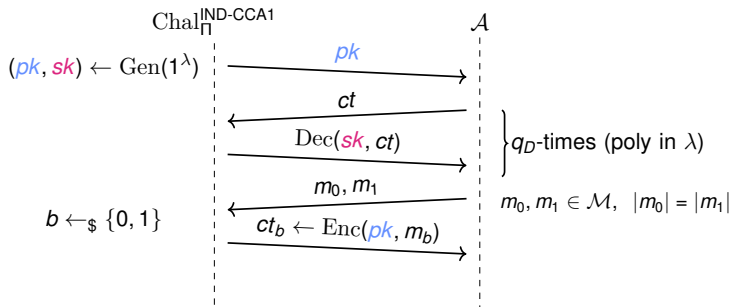
# IND-CCA1 security experiment

Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.



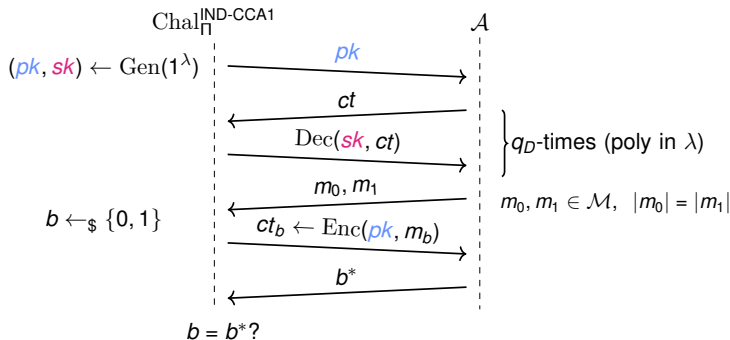
# IND-CCA1 security experiment

Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.



# IND-CCA1 security experiment

Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.



$\mathcal{A}$  wins iff  $b = b^*$

## IND-CCA1: Definition

### Definition 2.2: IND-CCA1 security

$\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  is Indistinguishable Under Chosen Ciphertext Attack 1 (*IND-CCA1 secure*) if for all PPT  $\mathcal{A}$  the advantage

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(\lambda) := 2 \cdot \left| \frac{1}{2} - \Pr[\mathcal{A} \text{ wins the IND-CCA1 experiment}] \right|$$

is negligible in  $\lambda$ .

## IND-CCA1: Definition

### Definition 2.2: IND-CCA1 security

$\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  is Indistinguishable Under Chosen Ciphertext Attack 1 (*IND-CCA1 secure*) if for all PPT  $\mathcal{A}$  the advantage

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(\lambda) := 2 \cdot \left| \frac{1}{2} - \Pr[\mathcal{A} \text{ wins the IND-CCA1 experiment}] \right|$$

is negligible in  $\lambda$ .

- $\text{IND-CCA1} \Rightarrow \text{IND-CPA}$  (ignore the oracle)

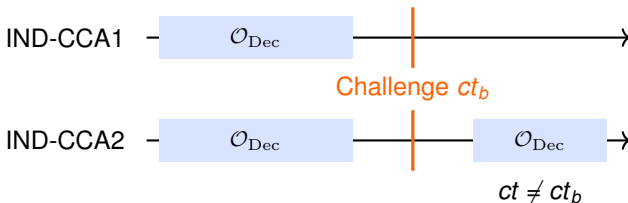
## Towards IND-CCA2

In practice the decryption device does not disappear after lunch.



## Towards IND-CCA2

In practice the decryption device does not disappear after lunch.



- $\mathcal{A}$  *may not query*  $\mathcal{O}_{\text{Dec}}(ct_b)$  — otherwise it just asks for  $m_b$ .
- This is the *only* restriction: every  $ct \neq ct_b$  is allowed.

# IND-CCA2 security experiment

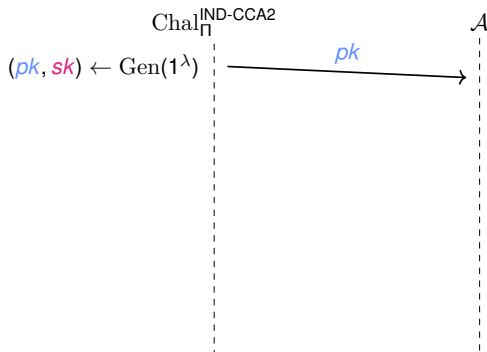
Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.

$\text{Chal}_{\Pi}^{\text{IND-CCA2}}$

$\mathcal{A}$

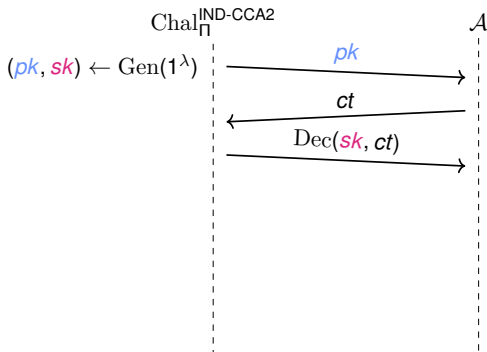
# IND-CCA2 security experiment

Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.



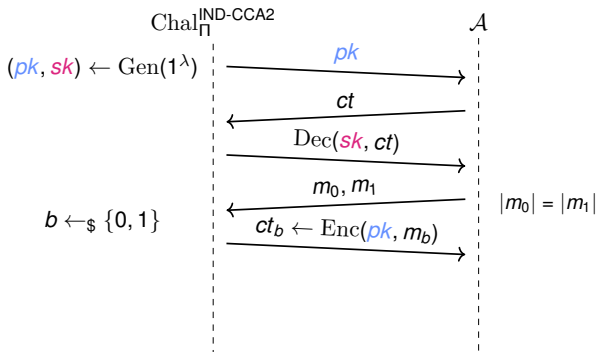
# IND-CCA2 security experiment

Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.



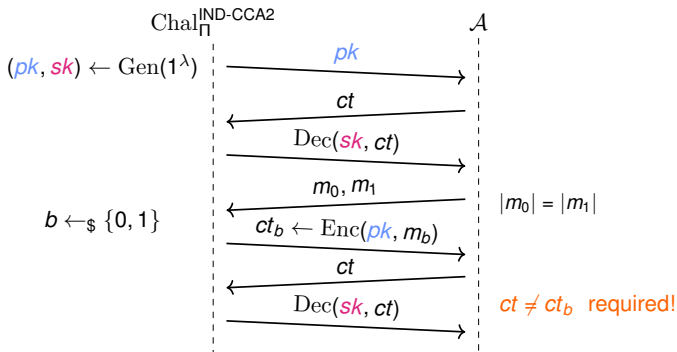
# IND-CCA2 security experiment

Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.



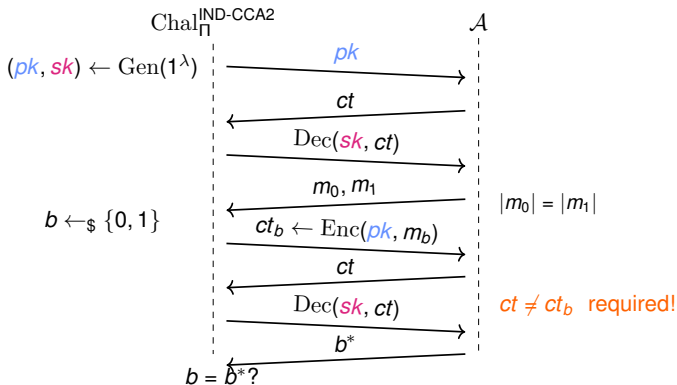
# IND-CCA2 security experiment

Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.



# IND-CCA2 security experiment

Let  $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  be an encryption scheme.



$\mathcal{A}$  wins iff  $b = b^*$

## IND-CCA2: Definition

### Definition 2.3: IND-CCA2 security

$\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  is *IND-CCA2 secure* if for all PPT  $\mathcal{A}$  the advantage

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CCA2}}(\lambda) := 2 \cdot \left| \frac{1}{2} - \Pr[\mathcal{A} \text{ wins the IND-CCA2 experiment}] \right|$$

is negligible in  $\lambda$ .

## IND-CCA2: Definition

### Definition 2.3: IND-CCA2 security

$\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  is *IND-CCA2 secure* if for all PPT  $\mathcal{A}$  the advantage

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CCA2}}(\lambda) := 2 \cdot \left| \frac{1}{2} - \Pr[\mathcal{A} \text{ wins the IND-CCA2 experiment}] \right|$$

is negligible in  $\lambda$ .

- $\text{IND-CCA2} \Rightarrow \text{IND-CPA1}$  (ignore the oracle after challenge query)

## IND-CCA2: Definition

### Definition 2.3: IND-CCA2 security

$\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$  is *IND-CCA2 secure* if for all PPT  $\mathcal{A}$  the advantage

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CCA2}}(\lambda) := 2 \cdot \left| \frac{1}{2} - \Pr[\mathcal{A} \text{ wins the IND-CCA2 experiment}] \right|$$

is negligible in  $\lambda$ .

- $\text{IND-CCA2} \Rightarrow \text{IND-CPA1}$  (ignore the oracle after challenge query)
- $\text{IND-CCA1} \not\Rightarrow \text{IND-CCA2}$  (easy exercise)

## IND-CCA2 security of ElGamal

Recall ElGamal:

$pk = h = g^x$ ,  $sk = x$  and ciphertexts are of the form

$$\text{Enc}(pk, m) = ct = (c_1 := g^r, c_2 := m \cdot h^r), \text{ for } r \leftarrow \mathbb{Z}_p.$$

# IND-CCA2 security of ElGamal

Recall ElGamal:

$pk = h = g^x$ ,  $sk = x$  and ciphertexts are of the form

$$\text{Enc}(pk, m) = ct = (c_1 := g^r, c_2 := m \cdot h^r), \text{ for } r \leftarrow \mathbb{Z}_p.$$

Attack on ElGamal, querying  $\mathcal{O}_{\text{Dec}}$  **after** the challenge

- Send  $m_0, m_1$ , receive  $ct_b = (c_1, c_2)$
- Query  $\mathcal{O}_{\text{Dec}}(c_1, c_2 \cdot m')$  for some  $m' \neq 1$  ( $\neq ct_b$ , so allowed)
- Receive  $m_b \cdot m' \Rightarrow$  recover  $m_b$
- Output the  $b^*$  with  $m_{b^*} = m_b$

# IND-CCA2 security of ElGamal

Recall ElGamal:

$pk = h = g^x$ ,  $sk = x$  and ciphertexts are of the form

$$\text{Enc}(pk, m) = ct = (c_1 := g^r, c_2 := m \cdot h^r), \text{ for } r \leftarrow \mathbb{Z}_p.$$

Attack on ElGamal, querying  $\mathcal{O}_{\text{Dec}}$  **after** the challenge

- Send  $m_0, m_1$ , receive  $ct_b = (c_1, c_2)$
- Query  $\mathcal{O}_{\text{Dec}}(c_1, c_2 \cdot m')$  for some  $m' \neq 1$  ( $\neq ct_b$ , so allowed)
- Receive  $m_b \cdot m' \Rightarrow$  recover  $m_b$
- Output the  $b^*$  with  $m_{b^*} = m_b$

- Advantage = 1 (exploits malleability of ElGamal)
- It needs a query *after* seeing  $ct \Rightarrow$  not an IND-CCA1 attack.

# IND-CCA1 security of ElGamal

- Is ElGamal IND-CCA1 secure?

# IND-CCA1 security of ElGamal

- Is ElGamal IND-CCA1 secure?
- We don't know! Open question since 1990.

# IND-CCA1 security of ElGamal

- Is ElGamal IND-CCA1 secure?
- We don't know! Open question since 1990.
  - No attack known, but also no proof from DDH known.
  - Schäge (EUROCRYPT'24): cannot be prove IND-CCA1 secure by “black-box reductions”

Intuitively this means that “no standard proof techniques won't work”.

# IND-CCA1 security of ElGamal

- Is ElGamal IND-CCA1 secure?
- We don't know! Open question since 1990.
  - No attack known, but also no proof from DDH known.
  - Schäge (EUROCRYPT'24): cannot be prove IND-CCA1 secure by “black-box reductions”

Intuitively this means that “no standard proof techniques won't work”.

- Version of ElGamal IND-CCA1 secure (Damgård's ElGamal)

# Summary of the notions



## Summary of the notions



ElGamal: IND-CPA **yes**    IND-CCA1 ?    IND-CCA2 **no**

## Summary of the notions



ElGamal: IND-CPA **yes**    IND-CCA1 ?    IND-CCA2 **no**

**Next lectures:** generic transformations    IND-CPA  $\longrightarrow$  IND-CCA2

# The difficulty: decryption queries

Attempt:  $\Pi$  IND-CPA scheme  $\Rightarrow \Pi$  IND-CCA1 secure.

Chal $_{\Pi}^{\text{IND-CPA}}$

$\mathcal{B}$

$\mathcal{A}$

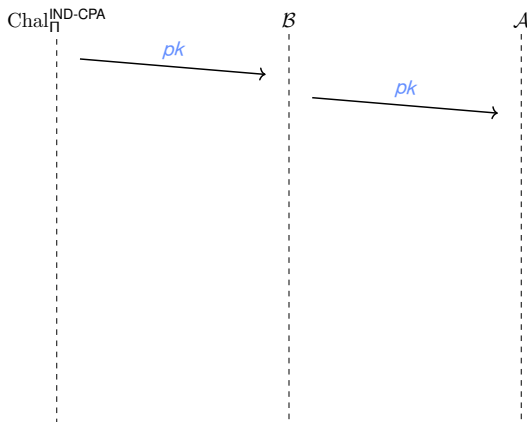
-----

-----

-----

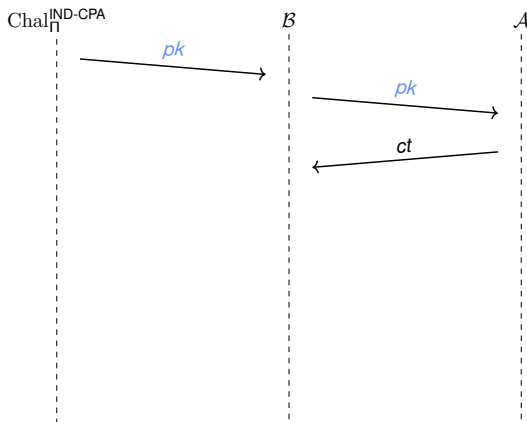
# The difficulty: decryption queries

Attempt:  $\Pi$  IND-CPA scheme  $\Rightarrow \Pi$  IND-CCA1 secure.



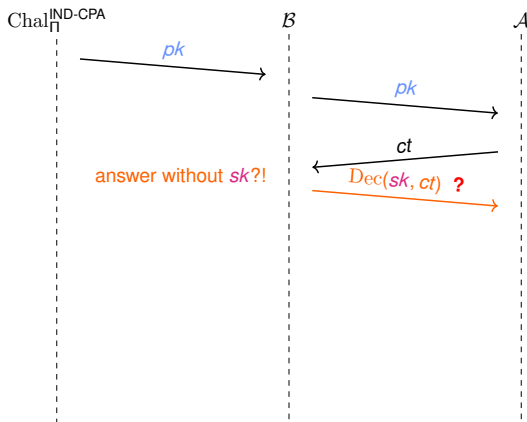
# The difficulty: decryption queries

Attempt:  $\Pi$  IND-CPA scheme  $\Rightarrow$   $\Pi$  IND-CCA1 secure.



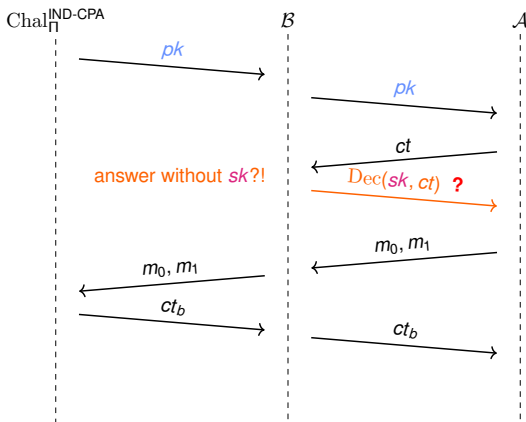
# The difficulty: decryption queries

Attempt:  $\Pi$  IND-CPA scheme  $\Rightarrow$   $\Pi$  IND-CCA1 secure.



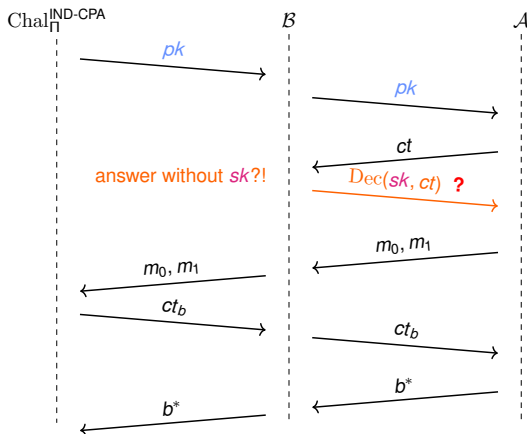
# The difficulty: decryption queries

Attempt:  $\Pi$  IND-CPA scheme  $\Rightarrow$   $\Pi$  IND-CCA1 secure.



# The difficulty: decryption queries

Attempt:  $\Pi$  IND-CPA scheme  $\Rightarrow \Pi$  IND-CCA1 secure.



# How the course resolves it

Three ways to answer decryption queries without *sk*:

- *Naor–Yung* (Zero-Knowledge Proofs)  
encrypt twice + prove consistency  $\Rightarrow$  decrypt with the *other* key.

# How the course resolves it

Three ways to answer decryption queries without *sk*:

- *Naor–Yung* (Zero-Knowledge Proofs)  
encrypt twice + prove consistency  $\Rightarrow$  decrypt with the *other* key.
- *Cramer–Shoup* (Universal Hash Proofs)  
ciphertexts carry a checkable tag  $\Rightarrow$  reject invalid ones without *sk*.

# How the course resolves it

Three ways to answer decryption queries without *sk*:

- *Naor–Yung* (Zero-Knowledge Proofs)  
encrypt twice + prove consistency  $\Rightarrow$  decrypt with the *other* key.
- *Cramer–Shoup* (Universal Hash Proofs)  
ciphertexts carry a checkable tag  $\Rightarrow$  reject invalid ones without *sk*.
- *Fujisaki–Okamoto* (Random Oracle Model)  
 $\mathcal{B}$  sees  $\mathcal{A}$ 's hash queries  $\Rightarrow$  it already knows the plaintexts.